Read-Through

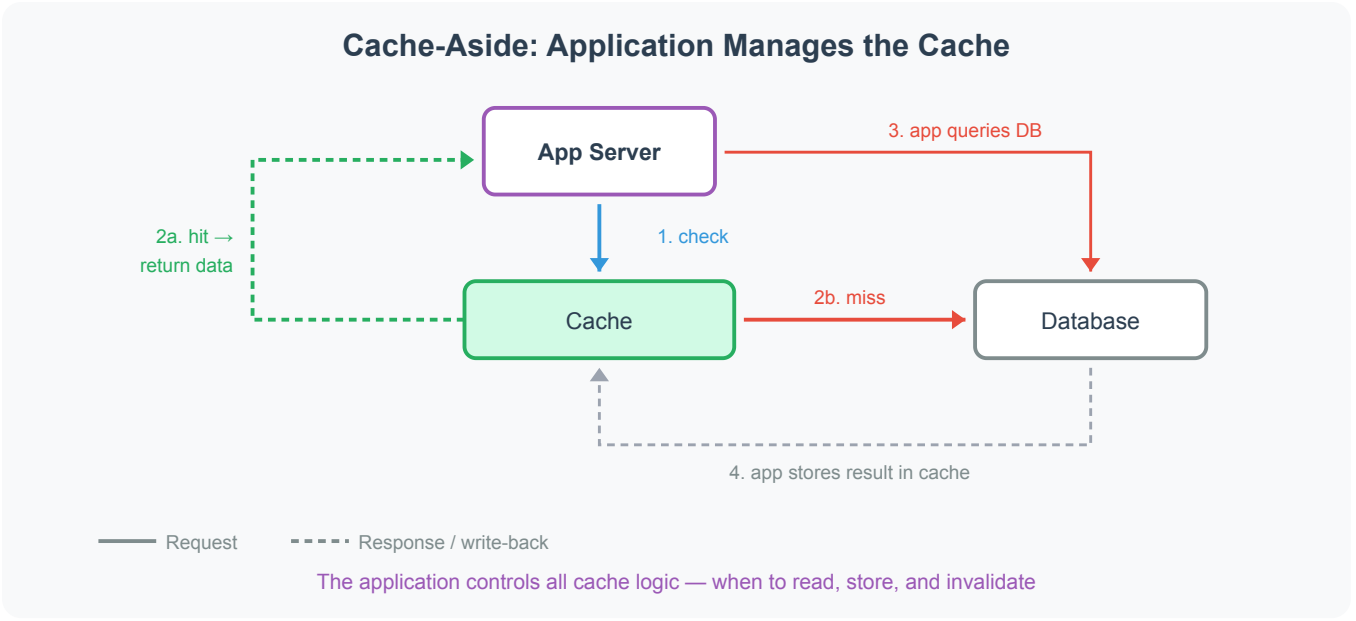
systemdesignschool.io/fundamentals/caching-read-patterns

Read Patterns: Fetching Data

You have a cache, structured keys, and data in the database. The question now: who is responsible for populating the cache? When a request arrives and the cache is empty, something must fetch from the database and store the result. The two main patterns differ in where that responsibility lives.

Cache-Aside (Lazy Loading)

In **cache-aside**, the application manages all cache interactions explicitly. The application code checks the cache, handles misses by querying the database, and writes results back to the cache.



The sequence on every read:

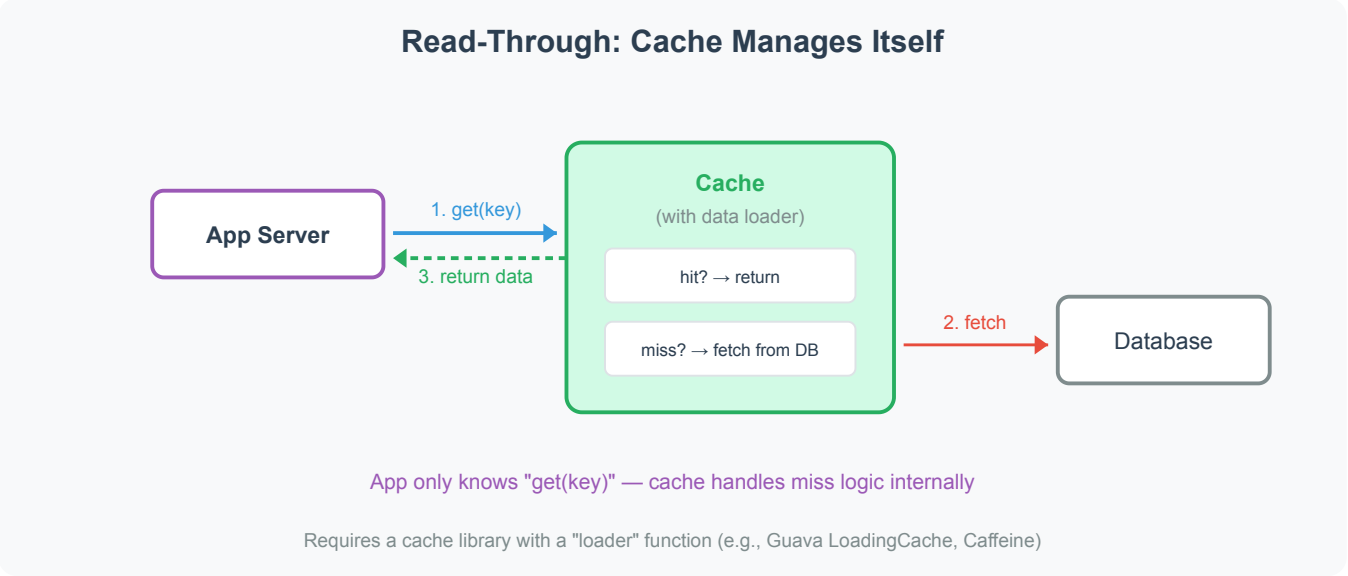
1. Application checks the cache for the key
2. **Hit:** return the cached value directly
3. **Miss:** application queries the database
4. Application writes the result into the cache
5. Application returns the result to the caller

The application owns the entire flow. The cache is a passive data store—it doesn't know anything about the database. This is the most common pattern in production systems.

Strengths. The application controls exactly what gets cached. Only data that is actually requested enters the cache (no wasted memory on unused entries). The cache layer is simple—any key-value store works. If the cache goes down, the application falls back to the database directly.

Weakness. Cache logic spreads across the codebase. Every data access point needs the same check-cache → miss → query-DB → store pattern. This creates duplication unless you abstract it into a helper function.

In **read-through**, the cache itself handles misses. The application simply asks the cache for data. If the cache doesn't have it, the cache fetches from the database using a configured "loader" function.



The sequence on every read:

1. Application calls `cache.get(key)`
2. Cache checks its store. **Hit:** return immediately
3. **Miss:** cache calls the loader function, which queries the database
4. Cache stores the result and returns it to the application

The application only knows `get(key)`. It never interacts with the database for cached data types. The cache library handles miss logic, storage, and expiration internally.

Strengths. Application code is cleaner—no cache-miss handling at each call site. The cache handles misses consistently across all call sites. Some implementations coalesce concurrent misses for the same key (preventing the thundering herd problem where, say, 100 requests all query the database for the same missing entry).

Weakness. Requires a cache library that supports loader functions (e.g., Guava LoadingCache, Caffeine in Java, or custom wrappers). The cache layer is more complex—it now has a dependency on the database. If the loader fails, the cache must handle errors rather than just returning "miss."

When to Use Which

Consideration	Cache-Aside	Read-Through
Simplicity of cache layer	Simple (passive store)	Complex (needs loader config)
Application code	More verbose (miss handling everywhere)	Cleaner (just `get(key)`)
Cache failure behavior	App falls back to DB naturally	Must handle loader errors

Consideration	Cache-Aside	Read-Through
Concurrent miss protection	Manual (you build it)	Often built-in (request coalescing)
Technology requirements	Any key-value store	Cache library with loader support

Most distributed systems use **cache-aside** because it works with any cache backend (Redis, Memcached, or a simple in-process map) and keeps the cache layer generic and DB-unaware. The application retains full control over loading logic.

Read-through shines in applications with many data access points where duplicating miss-handling logic would be error-prone. It's common in monolithic Java applications using Guava or Caffeine, where the cache is in-process and the loader function is just a method call.

In system design interviews, default to cache-aside unless asked specifically about read-through. Most architectural diagrams assume the application manages cache interactions.

This is implemented in a hands-on lab showing the mechanics of check → miss → store in working code.